Software Requirements Specification (SRS) Template

Project: Smart Expense & Budget Management Application
Version: 1.0
Authors: Viren James Fernandes (PES2UG24CS590), YV Prateek (PES2UG24CS617), Vikranth Acharyya (PES2UG24CS597), Vidushi Singh (PES2UG24CS581)
Date: 07-09-2026
Status: Model Answer / Approved
Team No:3

Revision history

Version	Date	Author	Change summary	Approval
1.0	07-09-2026	Team 3	SRS with diagrams embedded	Pending

Approvals

Role	Name	Signature / Email	Date
Course Coordinator	Prof. Pavithra M		07-09-2026

Table of Contents

1. Introduction
2. Overall description
3. External interfaces
4. System features (detailed)
5. Non-functional requirements (detailed)
6. Quality attributes & Acceptance tests
7. UML Use-Case Diagram
8. Requirements Traceability Matrix (RTM)

1. Introduction

1.1 Purpose

This document is a Software Requirements Specification (SRS) for a Smart Expense & Budget Management Application. It defines the functional and non-functional requirements, interfaces, system features, and verification criteria for the application. The document is intended to serve as a reference for developers, testers, project stakeholders, and assessment evaluators.

1.2 Scope

The Smart Expense & Budget Management Application helps users record and manage their income and expenses, categorize transactions, create and monitor budgets, track spending patterns, and receive budget-related alerts. The system provides dashboards and reports to help users understand their financial activities and make informed budgeting decisions.

The application covers user account management, expense and income tracking, budget management, financial summaries, notifications, and report generation. It does not include direct banking transactions, loan processing, investment management, or financial transactions with external banks.

1.3 Audience

Developers, QA Engineers, System Integrators, Maintenance Technicians, and Assessment Evaluators.

1.4 Definitions

SRS – Software Requirements Specification

- UI – User Interface
- API – Application Programming Interface
- DB – Database
- OTP – One-Time Password
- CRUD – Create, Read, Update, Delete
- NFR – Non-Functional Requirement
- FR – Functional Requirement
- RTM – Requirements Traceability Matrix

2. Overall description

2.1 Product perspective

The Smart Expense & Budget Management Application is a personal financial management system that helps users record, organize, and monitor their income and expenses. The application provides a centralized interface for managing financial transactions, setting budgets, tracking spending, and viewing financial summaries. It consists of a user interface, application logic, database, and notification/reporting services.

2.2 Major product functions (detailed)

User registration and login

- User profile management
- Add, edit, and delete income transactions
- Add, edit, and delete expense transactions
- Categorize income and expenses
- Create and manage monthly/weekly budgets
- Track spending against budget limits
- Display remaining budget and available balance
- Generate daily, weekly, and monthly expense summaries
- Provide graphical spending reports
- Search and filter transactions
- Set and manage savings goals
- Generate alerts when spending approaches or exceeds budget limits
- Manage recurring income and expenses
- Export expense and budget reports

2.3 User roles and characteristics (expanded)

User: The primary user of the application who records income and expenses, creates budgets, monitors spending, and views financial reports. The user is expected to have basic knowledge of personal finance and smartphone/web applications.

Administrator: Responsible for managing user accounts, monitoring system activity , managing categories, and maintaining the application.

System: Automatically performs functions such as calculating budgets, generating summaries, monitoring spending limits, and sending notifications.

2.4 Operating environment

The application shall operate on desktop computers, laptops, tablets, and smartphones through a web browser or supported mobile interface. The

system shall use a server-side application, database server, and network connection for storing and retrieving user financial data.

2.5 Constraints

The system shall require user authentication to access to personal information.

- User financial data shall be protected from unauthorized access.
- The application shall depend on a database for storing transactions, budgets, and user information.
- The system shall require network connectivity for cloud/server-based features.
- The application shall maintain data consistency when transactions are added, modified, or deleted.
- The system shall comply with applicable data privacy and security requirements.
- The system shall not directly perform banking transactions or access users' bank accounts unless an authorized banking API is explicitly integrated.

3. External interface requirements

3.1 User interfaces

The primary user interface shall be a responsive web or mobile interface with a clear navigation structure.

- Dashboard: Displays total income, total expenses, remaining budget, and spending summaries.
- Expense and Income Entry: Provides forms to add, edit, and delete financial transactions.
- Budget Management: Allows users to create budgets, set spending limits, and monitor budget usage.
- Reports: Displays graphical and tabular summaries of income, expenses, and spending patterns.
- Notifications: Displays alerts when users approach or exceed their budget limits.
- Accessibility: The interface shall support readable text, clear labels, sufficient contrast, and keyboard navigation where applicable.

3.2 Hardware interfaces

The application does not require specialized hardware interfaces.

- Desktop/Laptop: Keyboard, mouse, and display.
- Mobile Devices: Touchscreen and display.
- Optional: Device notification services for budget alerts.

3.3 Software interfaces

- Database Management System: Stores user accounts, income, expenses, budgets, categories, and savings goals.
- Application Server: Processes user requests, performs calculations, and manages application logic.
- Authentication Service: Handles user registration, login, and authentication.
- Notification Service: Sends budget alerts and reminders.
- Optional Third-Party Services: Email or cloud storage services for notifications and report backup, if integrated.

3.4 Communications

- The application shall use HTTPS for secure communication between the client and server.
- The application shall use REST APIs or equivalent communication mechanisms for data exchange.
- Data exchanged between the application and server shall be transmitted in a structured format such as JSON.
- The system shall handle network failures and display appropriate error messages to users.

4. System features (detailed)

4.1 User Authentication and Management

Req ID	Requirement	Type	Priority	Source/Stakeholder	Acceptance criteria /	Comments /

					Test case ref	Dependencies
SEA-F-001	The system shall allow a new user to create an account using a valid name, email address, and password.	Functional	High	User / Security	AC-SEA-F-001: Valid registration details create a new user account. Test: TC-Auth-01	Requires user database
SEA-F-002	The system shall authenticate users using their registered email address and password.	Functional	High	User / Security	AC-SEA-F-002: Valid credentials allow login; invalid credentials are rejected. Test: TC-Auth-02	Requires authentication module
SEA-F-003	The system shall provide a password reset mechanism for users who forget their password.	Functional	Medium	User/Security	AC-SEA-F-003: User can request a password reset and successfully set a new password. Test: TC-Auth-03	Requires email/OTP service if implemented
SEA-F-004	The system shall allow authenticated users to update their	Functional	Medium	User	AC-SEA-F-004: Updated profile information is saved	Requires user database

	profile information.				and displayed correctly. Test: TC-Auth-04	

4.2 Cash Withdrawal

Description: Dispense cash in validated denominations and ensure ledger consistency with bank core.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
ATM-F-010	The system shall verify sufficient available balance and daily limit before dispensing cash.	Functional	High	Business	AC-ATM-F-010: Dispense occurs only if balance >= amount and within limits. Test: TC-WD-01	Core API check required
ATM-F-011	The system shall dispense requested amount using available denominations and update cash cassette counts.	Functional	High	Operations	AC-ATM-F-011: Dispensed amount equals requested; cassette counts decremented. Test: TC-WD-02	Hardware interaction with dispenser

4.3 Cash/Check Deposit

Requirements: The system shall accept deposits (envelope or bulk acceptor), validate amounts where possible, and queue offline transactions if network unavailable. IDs: ATM-F-020..ATM-F-025.

4.4 Funds Transfer

Requirements: Support intra-bank transfers between own accounts with confirmation and receipt. Reject transfers exceeding configured limits. IDs ATM-F-030..ATM-F-032.

4.5 Maintenance & Monitoring

Requirements: Provide a technician menu for diagnostics, remote monitoring API integration, secure admin authentication, and encrypted log shipping. IDs ATM-F-040..ATM-F-045.

5. Non-functional requirements (detailed)

NFRs below are measurable and tied to test plans. IDs ATM-NF-###.

Req ID	Requirement	Category	Priority	Acceptance criteria / Measurement
ATM-NF-001	Overall transaction response (from card insertion to end) shall be ≤ 5 seconds for 90% of transactions under normal load.	Performance	High	90th percentile ≤ 5s in production-like test. Test: TC-Perf-01
ATM-NF-002	System shall provide 99.9% availability monthly; scheduled maintenance windows excluded.	Reliability	High	Uptime reports show ≥99.9% per month. Test: Ops reports.

ATM-NF-003	All sensitive cardholder data must comply with PCI-DSS; PAN must not be stored in plaintext.	Security/Compliance	High	PCI-DSS audit checklist pass. Test: TC-Sec-01
ATM-NF-004	System shall provide logs with timestamped events retained for 7 years for audit purposes.	Audit/Data Retention	High	Automated archival and retrieval verified. Test: TC-OPS-01
ATM-NF-005	System shall support accessibility features (screen reader, high contrast) conforming to WCAG 2.1 AA where applicable.	Usability/Accessibility	Medium	Accessibility audit pass. Test: TC-UX-01

5.1. Security

5.1.1 Security Objectives

Security is important because the application stores personal financial information. The system shall make sure that users can access only their own information and that sensitive data is protected.

1. Protect users' financial information from unauthorized access or disclosure.
2. Ensure that only authenticated users can view or modify their own financial records.
3. Protect account credentials and data while they are stored and transmitted.
4. Keep important security and account-management activities traceable through appropriate logs.

5.1.2 Security Requirements

Req ID	Requirement (shall...)	Type	Priority	Acceptance criteria / Test case ref

SEA-SR-001	The system shall use HTTPS/TLS for communication between the client and server.	Security	High	No sensitive application traffic is sent over unencrypted HTTP. Test: TC-Sec-01
SEA-SR-002	The system shall store passwords using a strong one-way password-hashing mechanism and shall never store passwords as plain text.	Security	High	Database inspection confirms passwords are hashed and authentication works correctly. Test: TC-Sec-02
SEA-SR-003	The system shall prevent a user from viewing, editing, or deleting another user's transactions, budgets, or savings goals.	Authorization	High	Cross-user access attempts are denied. Test: TC-Sec-03
SEA-SR-004	The system shall validate user input on the server side to reduce risks such as injection and malformed requests.	Security	High	Security test inputs are rejected or safely handled without database/query manipulation. Test: TC-Sec-04
SEA-SR-005	The system shall automatically expire authenticated sessions after a period of inactivity.	Security	Medium	Inactive session cannot access protected screens after timeout. Test: TC-Sec-05

6. Quality attributes & Acceptance tests

6.1 Quality Attributes

The Smart Expense & Budget Management Application shall maintain the following quality attributes:

- **Performance:** The application shall provide a responsive user experience when users add, edit, delete, search, or view financial transactions and budgets.
- **Reliability:** The system shall maintain data consistency when income, expenses, budgets, savings goals, or recurring transactions are added, modified, or deleted.
- **Security:** User financial information and account credentials shall be protected from unauthorized access. Users shall only be able to access and manage their own transactions, budgets, and savings goals.
- **Usability:** The application shall provide a clear and responsive interface that allows users to easily record transactions, manage budgets, view summaries, and access reports.
- **Accessibility:** The user interface shall support readable text, clear labels, sufficient contrast, and keyboard navigation where applicable.
- **Availability:** The application shall remain available to authenticated users except during scheduled maintenance or unavoidable network or server failures.
- **Maintainability:** The system shall be organized into separate components for the user interface, application logic, database management, authentication, notifications, and reporting to simplify future maintenance and updates.

6.2 Acceptance Tests

The following acceptance test suites shall be used to verify that the system meets its functional and non-functional requirements:

Test Suite	Purpose
Authentication Tests	Verify user registration, login, password reset, and profile management.
Income and Expense Tests	Verify that users can add, edit, delete, categorize, search, and filter income and expense transactions.
Budget Management Tests	Verify budget creation, modification, spending tracking, remaining budget calculation, and budget limit monitoring.
Savings Goal Tests	Verify creation and management of savings goals.
Recurring Transaction Tests	Verify that recurring income and expense transactions are managed correctly.

Test Suite	Purpose
Financial Summary and Report Tests	Verify daily, weekly, and monthly summaries and graphical spending reports.
Notification Tests	Verify alerts when spending approaches or exceeds budget limits.
Export Tests	Verify that expense and budget reports can be exported successfully.
Security Tests	Verify HTTPS/TLS communication, password hashing, authorization, server-side input validation, and session expiry.
Accessibility Tests	Verify readable text, clear labels, sufficient contrast, and keyboard navigation where applicable.

6.3 Exit Criteria for Acceptance

The system shall be accepted when:

- All high-priority functional requirements have been implemented and successfully verified.
- User authentication and authorization requirements have been successfully tested.
- Income and expense transactions can be correctly created, edited, deleted, categorized, searched, and filtered.
- Budgets and savings goals can be created and managed correctly.
- Financial summaries, reports, and budget alerts function correctly.
- No critical security or non-functional requirement failures remain.
- The Requirements Traceability Matrix shows that all applicable requirements have corresponding test cases and have passed acceptance testing.

7. System models and diagrams

7.1 UML Use-Case diagram

The UML Use-Case Diagram shall represent the interactions between the main actors and the Smart Expense & Budget Management Application.

Actors

1. **User**
    - Register account
    - Log in
    - Reset password

- Manage profile
- Add income
- Add expense
- Edit or delete transactions
- Categorize transactions
- Search and filter transactions
- Create and manage budgets
- Monitor spending against budgets
- View remaining budget and available balance
- View financial summaries
- Generate and view reports
- Manage savings goals
- Manage recurring income and expenses
- Receive budget alerts
- Export expense and budget reports

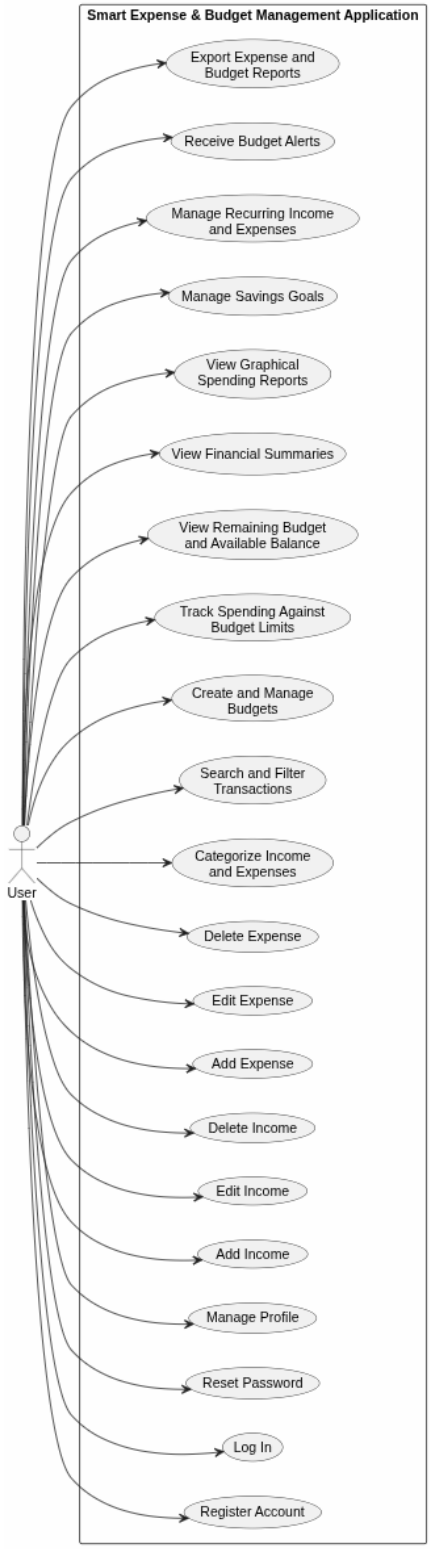
2. **Administrator**
    - Manage user accounts
    - Monitor system activity
    - Manage transaction categories
    - Maintain the application
3. **System**
    - Calculate available balance
    - Calculate remaining budget
    - Generate daily, weekly, and monthly financial summaries
    - Monitor spending against budget limits
    - Generate budget-related alerts
    - Process recurring transactions

Use-Case Diagram 1: User Financial Management

The first UML Use-Case Diagram shall show the **User** interacting with the following use cases:

- Register Account
- Log In
- Reset Password
- Manage Profile
- Manage Income
- Manage Expenses
- Categorize Transactions
- Search and Filter Transactions
- Manage Budgets
- Monitor Spending
- Manage Savings Goals
- Manage Recurring Transactions

- View Financial Summaries
- View Spending Reports
- Export Reports
- Receive Budget Alerts



Use-Case Diagram 2: Administration and Automated System Functions

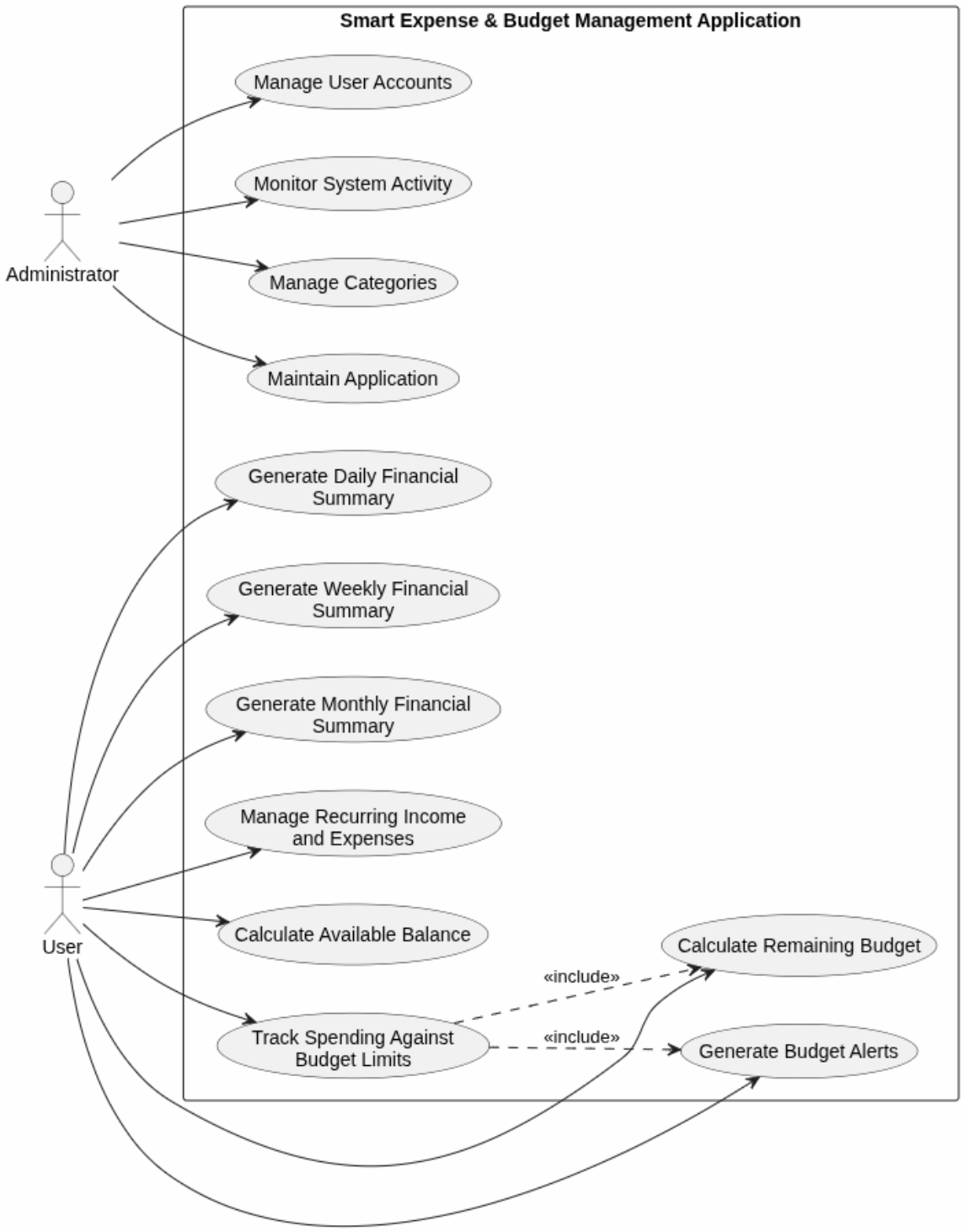
The second UML Use-Case Diagram shall show the **Administrator** and **System** interactions.

The Administrator shall interact with:

- Manage User Accounts
- Monitor System Activity
- Manage Categories
- Maintain Application

The System shall perform:

- Calculate Budget Usage
- Calculate Remaining Budget
- Calculate Available Balance
- Generate Financial Summaries
- Monitor Budget Limits
- Generate Budget Alerts
- Process Recurring Transactions



8. Requirements Traceability Matrix (RTM)

Req ID	Requirement Short Description	Section Ref / Design Spec	Module	Test Case(s)	Status	Comments
SEA-F-001	User registration using name, email, and password	4.1 / DS-Auth-01	Authentication Module	TC-Auth-01	N	Requires user database
SEA-F-002	Authenticate users using registered email and password	4.1 / DS-Auth-02	Authentication Module	TC-Auth-02	N	Invalid credentials must be rejected
SEA-F-003	Password reset mechanism	4.1 / DS-Auth-03	Authentication Module	TC-Auth-03	N	Email/OTP service may be required
SEA-F-004	Update user profile information	4.1 / DS-Auth-04	User Profile Module	TC-Auth-04	N	Changes must be saved correctly
SEA-F-005	Add, edit, and delete income transactions	4.2 / DS-Income-01	Income Module	TC-Income-01, TC-Income-02	N	CRUD operations required
SEA-F-006	Add, edit, and delete expense transactions	4.3 / DS-Expense-01	Expense Module	TC-Expense-01, TC-Expense-02	N	CRUD operations required
SEA-F-007	Categorize income and expense transactions	4.4 / DS-Category-01	Category Module	TC-Cat-01	N	Categories must be associated with transactions
SEA-F-008	Create and manage weekly/monthly budgets	4.5 / DS-Budget-01	Budget Module	TC-Budget-01, TC-Budget-02	N	Budget limits must be stored
SEA-F-	Track spending against budget limits	4.5 / DS-Budget-02	Budget Monitoring Module	TC-Budget-03	N	Spending must be compared with budget

009						
SEA-F-010	Display remaining budget and available balance	4.5 / DS-Budget-03	Dashboard Module	TC-Budget-04	N	Values calculated automatically
SEA-F-011	Generate daily, weekly, and monthly expense summaries	4.6 / DS-Report-01	Summary Module	TC-Report-01	N	Summary calculations must be accurate
SEA-F-012	Display graphical spending reports	4.6 / DS-Report-02	Reporting Module	TC-Report-02	N	Charts must reflect transaction data
SEA-F-013	Search and filter transactions	4.3 / DS-Search-01	Transaction Module	TC-Search-01	N	Search/filter results must be relevant
SEA-F-014	Create and manage savings goals	4.7 / DS-Saving-01	Savings Goal Module	TC-Saving-01	N	Goal progress must be tracked
SEA-F-015	Generate alerts when spending approaches/exceeds budget	4.8 / DS-Notify-01	Notification Module	TC-Notify-01	N	Alerts triggered based on budget usage
SEA-F-016	Manage recurring income and expenses	4.9 / DS-Recurring-01	Recurring Transaction Module	TC-Recurring-01	N	Recurring transactions processed automatically
SEA-F-017	Export expense and budget reports	4.6 / DS-Export-01	Report Export Module	TC-Export-01	N	Exported report must contain correct data
SEA-SR-001	Use HTTPS/TLS for client-server communication	5.1.2 / DS-Sec-01	Security Module	TC-Sec-01	N	Prevent unencrypted sensitive traffic
SEA-SR-	Store passwords using one-way hashing	5.1.2 / DS-Sec-02	Authentication Module	TC-Sec-02	N	Plain-text passwords must never be stored

002						
SEA-SR-003	Prevent users from accessing other users' data	5.1.2 / DS-Sec-03	Authorization Module	TC-Sec-03	N	Cross-user access must be denied
SEA-SR-004	Validate user input on the server side	5.1.2 / DS-Sec-04	Security/API Module	TC-Sec-04	N	Protect against injection/malformed requests
SEA-SR-005	Expire authenticated sessions after inactivity	5.1.2 / DS-Sec-05	Session Management Module	TC-Sec-05	N	Expired sessions cannot access protected pages
SEA-NF-001	Provide responsive performance for normal operations	5 / DS-Perf-01	Web UI / Application Server	TC-Perf-01	N	Response should meet defined performance target
SEA-NF-002	Maintain system availability	5 / DS-Avail-01	Application Server	TC-Avail-01	N	Scheduled maintenance excluded
SEA-NF-003	Maintain data consistency	5 / DS-Rel-01	Database Module	TC-Rel-01	N	No inconsistent transaction/budget data
SEA-NF-004	Support accessibility requirements	5 / DS-UX-01	User Interface	TC-UX-01	N	Readable text, labels, contrast, keyboard navigation
SEA-NF-005	Maintain modular and maintainable architecture	5 / DS-Maint-01	System Architecture	TC-Maint-01	N	Components separated by responsibility